IE-494 Lab 05 — YAFIM (Spark RDD)

> **Course:** IE-494 Big Data Processing · Autumn 2026
> **Instructor:** PM Jat (`pm_jat@dau.ac.in`)
> **Student:** Aditya Vaish · ID `202401401`
> **Submitted:** 11 September 2026
> **Constraint:** Spark **RDD API only**. No DataFrames, Datasets, or Spark SQL.

1. Assignment

Implement **basic YAFIM** (Yet Another Frequent Itemset Mining) on Spark RDDs:

- Paper Algorithm 2 — frequent 1-itemsets
- Paper Algorithm 3 — frequent k-itemsets for k ≥ 2 (apriori-gen + broadcast hash tree + `subset()`)
- Mine FIMI `retail.dat` with `minSupport = 1000`

Citation: Hongjian Qiu, Rong Gu, Chunfeng Yuan, Yihua Huang, *YAFIM: A Parallel Frequent Itemset Mining Algorithm with Spark*, IEEE International Parallel & Distributed Processing Symposium Workshops (IPDPSW 2014).

2. Dataset

property	value
transactions (baskets)	88,162
encoding	space-separated item IDs, one basket per line
minSupport	1000
relative support	1000 / 88162 ≈ 1.13%
Spark master	local[*] (Spark 4.2.0, spark-core_2.13, Java 17)
API	Java RDD only — no DataFrame / Dataset / Spark SQL

A tiny `data/toy.dat` (5 baskets) is used as a sanity check with `minSupport = 3`.

3. Algorithm

YAFIM is Apriori on Spark RDDs. The transaction RDD is parsed once, cached, and reused for every pass.

Phase I — Algorithm 2 (frequent 1-itemsets)

1. `flatMap` each cached transaction to `(item, 1)` pairs.
2. `reduceByKey` to support counts.
3. Keep items with count ≥ `minSupport`. That list is `L1`.

Phase II — Algorithm 3 (k ≥ 2)

For k = 2, 3, … until `Ck` or `Lk` is empty:

1. **apriori-gen:** self-join frequent (k−1)-itemsets on a shared (k−2)-prefix, then prune any k-candidate whose (k−1)-subsets are not all frequent.
2. Build an in-memory **hash tree** of `Ck` (branch factor 11, leaf split after 4 candidates) and **broadcast** it.
3. Each partition runs `subset()` / `HashTree.findCandidates` on every cached transaction: walk the tree by hashing successive items, and at leaves test `containsAll`.
4. `flatMap` matching candidates to `(key, 1)`, `reduceByKey`, keep counts ≥ `minSupport` → `Lk`.

`SparkContext.collect()` returns an unmodifiable list, so results are copied into `new ArrayList<>(...)` before `List.sort`.

Hash-tree support counting (bug fix)

Hashing two different items into the same bucket can visit a leaf more than once for the same transaction. Without dedupe, the same candidate was emitted multiple times per basket, which inflated support. Example before the fix: pair `39,48` counted **75856** (YAFIM) vs **29142** (lab3 MR-Apriori), and `|L2|` was **227** instead of **49**.

`findCandidates` now records keys in a `LinkedHashSet` and returns each candidate once per transaction. After the fix, YAFIM matches lab3 on every itemset key and count.

4. How to run

```
cd lab5
bash run_all.sh

# equivalent:
mvn -DskipTests package
mvn exec:java \
  -Dexec.mainClass=pmj.bdp.spark.lab5.Yafim \
  -Dexec.args="$(pwd)/data/retail.dat 1000 $(pwd)/output/yafim"
```

`mvn exec:java` does not recompile unless you also `package` / `compile`. Outputs: `output/yafim/L1.txt` … `L4.txt` and `frequent-itemsets.txt` (tab-separated `itemset\tsupport`).

5. Evaluation results

5.1 Toy sanity check (`toy.dat`, minSupport = 3)

5 transactions. Expected: L1 = items 1, 2, 3; three frequent pairs; L3 empty (the triple 1,2,3 has support 2).

itemset	support
1	4
2	4
3	4
1,2	3
1,3	3

itemset	support
2,3	3

L1 = 3, C2 = 3, L2 = 3, C3 = 1, L3 = 0. Matches the hand count.

5.2 Retail (`retail.dat`, minSupport = 1000)

transactions = 88162, relative minsup ≈ **1.13%**.

level	YAFIM Lk count	lab3 MR-Apriori Lk count	YAFIM Ck count	match?
L1	56	56	— (Algo-2)	yes
L2	49	49	1540	yes
L3	24	24	31	yes
L4	6	6	6	yes
L5	0 (stop)	0	0	yes

Total frequent itemsets written: **135** (56 + 49 + 24 + 6). Lab3's combined `frequent-itemsets.txt` lists k ≥ 2 only (79 itemsets). Every k ≥ 2 key and support in YAFIM equals lab3 (0 missing keys, 0 count mismatches).

source	support of 39,48
YAFIM after HashTree dedupe	29142
lab3 MR-Apriori	29142
YAFIM before dedupe (buggy)	75856

Singleton supports also match (`39` → 50675, `48` → 42135).

5.3 L4 — all 6 frequent 4-itemsets

itemset	support
32,38,39,48	1236
32,39,41,48	1646
36,38,39,48	1080
38,39,41,48	1991
38,39,48,110	1031
38,39,48,170	1193

5.4 L3 — all 24 frequent 3-itemsets

itemset	support
32,38,39	1840
32,38,48	1646
32,39,41	2359

itemset	support
32,39,48	5402
32,41,48	2063
36,38,39	1945
36,38,48	1360
36,39,48	1116
38,39,110	1740
38,39,170	2019
38,39,41	3051
38,39,48	6102
38,41,48	2374
38,48,110	1361
38,48,170	1538
39,41,48	7366
39,48,110	1037
39,48,170	1206
39,48,225	1400
39,48,237	1244
39,48,310	1347
39,48,475	1092
39,48,65	1797
39,48,89	2125

5.5 L2 — all 49 frequent 2-itemsets

itemset	support
32,38	2833
32,39	8455
32,41	3196
32,48	8034
36,38	2790
36,39	2037
36,48	1416
37,38	1046
38,110	2725
38,170	3031
38,286	1116

itemset	support
38,39	10345
38,41	3897
38,48	7944
39,101	1400
39,110	1759
39,1327	1156
39,147	1137
39,170	2059
39,2238	1287
39,225	2351
39,237	1929
39,255	1057
39,270	1194
39,271	1434
39,310	1852
39,41	11414
39,413	1130
39,438	1260
39,475	1500
39,48	29142
39,65	2787
39,79	1111
39,89	2749
41,48	9018
48,101	1311
48,110	1380
48,147	1036
48,170	1557
48,225	1736
48,237	1682
48,255	1057
48,271	1090
48,310	1692
48,413	1135
48,438	1025

itemset	support
48,475	1428
48,65	2529
48,89	2798

5.6 L1 — all 56 frequent 1-itemsets

item	support
9	1372
19	1005
32	15167
36	2936
37	1074
38	15596
39	50675
41	14945
48	42135
49	1120
60	1489
65	4472
78	1060
79	1600
89	3837
101	2237
110	2794
117	1026
123	1302
147	1779
161	1010
170	3099
185	1376
201	1133
225	3257
237	3032
249	1160
255	1474
270	1734

item	support
271	2094
286	1183
301	1204
310	2594
338	1274
413	1880
438	1863
475	2167
533	1487
548	1137
589	1119
592	1227
604	1209
677	1110
740	1181
824	1210
1004	1102
1146	1426
1327	1786
1393	1161
2238	1715
12925	1467
13041	1051
14098	1291
15832	1143
16010	1316
16217	1166

5.7 Terminal summary (retail)

```
transactions = 88162
minSupport   = 1000  (0.0113 of n)

======== YAFIM Algo-2  L1 ========
|L1| = 56

======== YAFIM Algo-3  L2 ========
|C2| = 1540
|L2| = 49

======== YAFIM Algo-3  L3 ========
|C3| = 31
```

```
|L3| = 24

======== YAFIM Algo-3  L4 ========
|C4| = 6
|L4| = 6

======== YAFIM Algo-3  L5 ========
|C5| = 0

Wrote 135 frequent itemsets to output/yafim/frequent-itemsets.txt

56  output/yafim/L1.txt
49  output/yafim/L2.txt
24  output/yafim/L3.txt
 6  output/yafim/L4.txt
135 output/yafim/frequent-itemsets.txt
```

6. Source code (complete, formatted)

All of `lab5/`. Spark RDD only. Line-numbered, syntax-highlighted Java / XML / bash.

Yafim.java

`lab5/src/main/java/pmj/bdp/spark/lab5/Yafim.java`

```java
package pmj.bdp.spark.lab5;

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.broadcast.Broadcast;

import scala.Tuple2;

/**
 * YAFIM — Yet Another Frequent Itemset Mining
 * (Qiu, Gu, Yuan, Huang; IEEE IPDPSW 2014).
 *
 * Spark RDD only (no DataFrame / SQL). Two phases from the paper:
 *   Algo-2  frequent 1-itemsets L1  (flatMap + reduceByKey + minsup filter)
 *   Algo-3  frequent k-itemsets, k ≥ 2 (apriori-gen, broadcast hash tree,
 *           subset() on each cached transaction, reduceByKey)
 *
 * Usage: Yafim &lt;transactions&gt; [minSupport] [outputDir]
 * Defaults: minSupport=1000, outputDir=output/yafim
 */
public class Yafim {

    public static void main(String[] args) throws IOException {
        if (args.length < 1) {
            System.err.println("Usage: Yafim <transactions> [minSupport] [outputDir]");
            System.exit(1);
        }
        String input = args[0];
        int minSupport = args.length > 1 ? Integer.parseInt(args[1]) : 1000;
        Path outDir = Path.of(args.length > 2 ? args[2] : "output/yafim");

        JavaSparkContext sc = SparkUtil.localContext("Lab5 YAFIM");
        try {
```

```
            run(sc, input, minSupport, outDir);
        } finally {
            sc.stop();
            sc.close();
        }
    }

    static void run(JavaSparkContext sc, String input, int minSupport, Path outDir)
            throws IOException {
        Files.createDirectories(outDir);

        JavaRDD<int[]> transactions = sc.textFile(input)
                .map(Itemset::parseTransaction)
                .filter(t -> t.length > 0)
                .cache();

        long n = transactions.count();
        System.out.println("transactions = " + n);
        System.out.println("minSupport   = " + minSupport
                + "  (" + String.format("%.4f", minSupport / (double) n) + " of n)");

        List<Tuple2<String, Integer>> allFrequent = new ArrayList<>();

        // ---- Algo-2: Phase I, frequent 1-itemsets ----
        System.out.println();
        System.out.println("======== YAFIM Algo-2  L1 ========");
        List<Tuple2<String, Integer>> l1kv = algo2Frequent1(transactions, minSupport);
        List<int[]> lk = keysToItemsets(l1kv);
        System.out.println("|L1| = " + lk.size());
        writeLevel(outDir.resolve("L1.txt"), l1kv);
        allFrequent.addAll(l1kv);

        // ---- Algo-3: Phase II, k >= 2 ----
        for (int k = 2; ; k++) {
            System.out.println();
            System.out.println("======== YAFIM Algo-3  L" + k + " ========");
            List<int[]> ck = Itemset.generateCandidates(lk);
            System.out.println("|C" + k + "| = " + ck.size());
            if (ck.isEmpty()) {
                break;
            }

            List<Tuple2<String, Integer>> lkkv = algo3FrequentK(sc, transactions, ck,
minSupport);
            lk = keysToItemsets(lkkv);
            System.out.println("|L" + k + "| = " + lk.size());
            if (lk.isEmpty()) {
                break;
            }
            writeLevel(outDir.resolve("L" + k + ".txt"), lkkv);
            allFrequent.addAll(lkkv);
        }

        Path combined = outDir.resolve("frequent-itemsets.txt");
        writeLevel(combined, allFrequent);
        System.out.println();
        System.out.println("Wrote " + allFrequent.size() + " frequent itemsets to " +
combined);
    }

    /**
     * Paper Algorithm 2 — Phase I.
     * Transactions RDD already cached. flatMap items to (item, 1),
     * reduceByKey, keep counts ≥ minsup.
     */
    static List<Tuple2<String, Integer>> algo2Frequent1(
            JavaRDD<int[]> transactions, int minSupport) {
        JavaPairRDD<Integer, Integer> counts = transactions
                .flatMapToPair(t -> {
                    List<Tuple2<Integer, Integer>> out = new ArrayList<>(t.length);
                    for (int item : t) {
```

```
                        out.add(new Tuple2<>(item, 1));
                    }
                    return out.iterator();
                })
                .reduceByKey(Integer::sum)
                .filter(kv -> kv._2 >= minSupport);

        List<Tuple2<String, Integer>> rows = new ArrayList<>();
        for (Tuple2<Integer, Integer> kv : counts.collect()) {
            rows.add(new Tuple2<>(Integer.toString(kv._1), kv._2));
        }
        rows.sort(Comparator.comparing(t -> Integer.parseInt(t._1)));
        return rows;
    }

    /**
     * Paper Algorithm 3 — Phase II for one k.
     * Build / broadcast a hash tree of Ck, run subset() on each transaction,
     * reduceByKey, keep counts ≥ minsup.
     */
    static List<Tuple2<String, Integer>> algo3FrequentK(
            JavaSparkContext sc,
            JavaRDD<int[]> transactions,
            List<int[]> candidates,
            int minSupport) {

        Broadcast<HashTree> treeBc = sc.broadcast(new HashTree(candidates));
        try {
            JavaPairRDD<String, Integer> counts = transactions
                    .flatMapToPair(t -> {
                        List<int[]> hits = treeBc.value().findCandidates(t);
                        List<Tuple2<String, Integer>> out = new ArrayList<>(hits.size());
                        for (int[] c : hits) {
                            out.add(new Tuple2<>(Itemset.toKey(c), 1));
                        }
                        return out.iterator();
                    })
                    .reduceByKey(Integer::sum)
                    .filter(kv -> kv._2 >= minSupport);

            List<Tuple2<String, Integer>> rows = new ArrayList<>(counts.collect());
            rows.sort(Comparator.comparing(t -> t._1));
            return rows;
        } finally {
            treeBc.destroy();
        }
    }

    private static List<int[]> keysToItemsets(List<Tuple2<String, Integer>> rows) {
        List<int[]> list = new ArrayList<>(rows.size());
        for (Tuple2<String, Integer> row : rows) {
            list.add(Itemset.parseKey(row._1));
        }
        return list;
    }

    private static void writeLevel(Path file, List<Tuple2<String, Integer>> rows)
            throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file)) {
            for (Tuple2<String, Integer> row : rows) {
                w.write(row._1);
                w.write('\t');
                w.write(Integer.toString(row._2));
                w.newLine();
            }
        }
    }
}
```

HashTree.java

lab5/src/main/java/pmj/bdp/spark/lab5/HashTree.java

```java
package pmj.bdp.spark.lab5;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;

/**
 * Apriori hash tree used by YAFIM Algo-3 {@code subset()} to find which
 * candidate k-itemsets a transaction contains (Qiu et al., IPDPSW 2014).
 *
 * Interior nodes hash on the next item; leaves store candidates. A leaf
 * splits once it exceeds {@link #MAX_LEAF} and depth &lt; k.
 */
public final class HashTree implements Serializable {

    private static final long serialVersionUID = 1L;
    private static final int BRANCH = 11;
    private static final int MAX_LEAF = 4;

    private final Node root;
    private final int k;

    public HashTree(List<int[]> candidates) {
        if (candidates == null || candidates.isEmpty()) {
            throw new IllegalArgumentException("candidates must be non-empty");
        }
        this.k = candidates.get(0).length;
        this.root = new Node(true);
        for (int[] c : candidates) {
            insert(root, c, 0);
        }
    }

    /**
     * Candidates contained in the (sorted, unique) transaction.
     * Each candidate is returned at most once: hash collisions can visit the
     * same leaf via multiple item prefixes, which would otherwise inflate support.
     */
    public List<int[]> findCandidates(int[] transaction) {
        LinkedHashSet<String> seen = new LinkedHashSet<>();
        List<int[]> hits = new ArrayList<>();
        search(root, transaction, 0, 0, seen, hits);
        return hits;
    }

    private void insert(Node node, int[] itemset, int depth) {
        if (node.leaf) {
            node.candidates.add(itemset);
            if (node.candidates.size() > MAX_LEAF && depth < k) {
                split(node, depth);
            }
            return;
        }
        int bucket = bucket(itemset[depth]);
        if (node.children[bucket] == null) {
            node.children[bucket] = new Node(true);
        }
        insert(node.children[bucket], itemset, depth + 1);
    }

    private void split(Node leaf, int depth) {
        List<int[]> old = leaf.candidates;
        leaf.leaf = false;
        leaf.candidates = null;
```

```
        leaf.children = new Node[BRANCH];
        for (int[] c : old) {
            insert(leaf, c, depth);
        }
    }

    private void search(Node node, int[] t, int depth, int start,
                        LinkedHashSet<String> seen, List<int[]> hits) {
        if (node == null) {
            return;
        }
        if (node.leaf) {
            for (int[] cand : node.candidates) {
                if (Itemset.containsAll(t, cand) && seen.add(Itemset.toKey(cand))) {
                    hits.add(cand);
                }
            }
            return;
        }
        int remaining = k - depth;
        for (int i = start; i <= t.length - remaining; i++) {
            Node child = node.children[bucket(t[i])];
            if (child != null) {
                search(child, t, depth + 1, i + 1, seen, hits);
            }
        }
    }

    private static int bucket(int item) {
        int m = item % BRANCH;
        return m < 0 ? m + BRANCH : m;
    }

    private static final class Node implements Serializable {
        private static final long serialVersionUID = 1L;
        boolean leaf;
        List<int[]> candidates;
        Node[] children;

        Node(boolean leaf) {
            this.leaf = leaf;
            if (leaf) {
                this.candidates = new ArrayList<>();
            } else {
                this.children = new Node[BRANCH];
            }
        }
    }
}
```

Itemset.java

`lab5/src/main/java/pmj/bdp/spark/lab5/Itemset.java`

```
package pmj.bdp.spark.lab5;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * Sorted integer itemset helpers (Apriori join + prune).
 */
public final class Itemset {

    private Itemset() {
    }
```

```java

    public static int[] parseTransaction(String line) {
        if (line == null || line.isBlank()) {
            return new int[0];
        }
        String[] tok = line.trim().split("\\s+");
        int[] raw = new int[tok.length];
        int n = 0;
        for (String t : tok) {
            if (t.isEmpty()) {
                continue;
            }
            raw[n++] = Integer.parseInt(t);
        }
        if (n == 0) {
            return new int[0];
        }
        Arrays.sort(raw, 0, n);
        int w = 1;
        for (int i = 1; i < n; i++) {
            if (raw[i] != raw[w - 1]) {
                raw[w++] = raw[i];
            }
        }
        return Arrays.copyOf(raw, w);
    }

    public static String toKey(int[] items) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < items.length; i++) {
            if (i > 0) {
                sb.append(',');
            }
            sb.append(items[i]);
        }
        return sb.toString();
    }

    public static int[] parseKey(String key) {
        String[] parts = key.split(",");
        int[] items = new int[parts.length];
        for (int i = 0; i < parts.length; i++) {
            items[i] = Integer.parseInt(parts[i].trim());
        }
        return items;
    }

    /** Both arrays sorted. */
    public static boolean containsAll(int[] transaction, int[] candidate) {
        int i = 0;
        int j = 0;
        while (i < transaction.length && j < candidate.length) {
            if (transaction[i] == candidate[j]) {
                i++;
                j++;
            } else if (transaction[i] < candidate[j]) {
                i++;
            } else {
                return false;
            }
        }
        return j == candidate.length;
    }

    /**
     * Apriori-gen: self-join L_{k-1} then prune candidates whose
     * (k-1)-subsets are not all frequent.
     */
    public static List<int[]> generateCandidates(List<int[]> previous) {
        List<int[]> candidates = new ArrayList<>();
        if (previous.isEmpty()) {
```

```
            return candidates;
        }
        int kMinus1 = previous.get(0).length;
        Set<String> frequent = new HashSet<>();
        for (int[] itemset : previous) {
            frequent.add(toKey(itemset));
        }

        for (int i = 0; i < previous.size(); i++) {
            int[] a = previous.get(i);
            for (int j = i + 1; j < previous.size(); j++) {
                int[] b = previous.get(j);
                if (!prefixEqual(a, b, kMinus1 - 1)) {
                    continue;
                }
                int lastA = a[kMinus1 - 1];
                int lastB = b[kMinus1 - 1];
                if (lastA == lastB) {
                    continue;
                }
                int[] cand = new int[kMinus1 + 1];
                System.arraycopy(a, 0, cand, 0, kMinus1 - 1);
                if (lastA < lastB) {
                    cand[kMinus1 - 1] = lastA;
                    cand[kMinus1] = lastB;
                } else {
                    cand[kMinus1 - 1] = lastB;
                    cand[kMinus1] = lastA;
                }
                if (allSubsetsFrequent(cand, frequent)) {
                    candidates.add(cand);
                }
            }
        }
        return candidates;
    }

    private static boolean prefixEqual(int[] a, int[] b, int len) {
        for (int i = 0; i < len; i++) {
            if (a[i] != b[i]) {
                return false;
            }
        }
        return true;
    }

    private static boolean allSubsetsFrequent(int[] candidate, Set<String> frequent) {
        int k = candidate.length;
        int[] subset = new int[k - 1];
        for (int drop = 0; drop < k; drop++) {
            int idx = 0;
            for (int i = 0; i < k; i++) {
                if (i != drop) {
                    subset[idx++] = candidate[i];
                }
            }
            if (!frequent.contains(toKey(subset))) {
                return false;
            }
        }
        return true;
    }
}
```

SparkUtil.java

`lab5/src/main/java/pmj/bdp/spark/lab5/SparkUtil.java`

```java
package pmj.bdp.spark.lab5;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaSparkContext;

public final class SparkUtil {

    private SparkUtil() {
    }

    public static JavaSparkContext localContext(String appName) {
        SparkConf conf = new SparkConf()
                .setAppName(appName)
                .setMaster("local[*]")
                .set("spark.ui.enabled", "false")
                .set("spark.driver.host", "127.0.0.1");
        JavaSparkContext sc = new JavaSparkContext(conf);
        sc.setLogLevel("ERROR");
        return sc;
    }
}
```

pom.xml

lab5/pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>pmj.bdp.spark</groupId>
    <artifactId>lab5</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
        <spark.version>4.2.0</spark.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.spark</groupId>
            <artifactId>spark-core_2.13</artifactId>
            <version>${spark.version}</version>
        </dependency>
    </dependencies>

    <build>
        <finalName>lab5</finalName>
        <plugins>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.5.0</version>
            </plugin>
        </plugins>
    </build>

</project>
```

run_all.sh

lab5/run_all.sh

```
#!/usr/bin/env bash
# Build and run Lab 05 YAFIM (Spark RDD). Do not use DataFrames/SQL.
set -euo pipefail

ROOT="$(cd "$(dirname "$0")" && pwd)"
cd "$ROOT"

RETAIL="$ROOT/data/retail.dat"
OUT="$ROOT/output/yafim"
MINSUP="${MINSUP:-1000}"

echo "==> Building..."
mvn -q -DskipTests package

rm -rf "$OUT"

echo
echo "==> YAFIM on retail.dat  minSupport=$MINSUP"
mvn -q exec:java \
  -Dexec.mainClass=pmj.bdp.spark.lab5.Yafim \
  -Dexec.args="$RETAIL $MINSUP $OUT"

echo
echo "---- summary ----"
wc -l "$OUT"/L*.txt "$OUT"/frequent-itemsets.txt
echo
echo "Done. Outputs under $OUT"
```